背景知识

分布式通信原语

- all-reduce：每一个GPU把数据发送给其余所有GPU，每个GPU对接收到的数据进行求和等操作
- reduce：将多个GPU的数据发送到指定GPU进行求和等操作
- broadcast：将一个GPU上的数据同时发送到所有其他GPU
- all-gather：每一个GPU把数据发送到其余所有GPU，每个GPU对于接受到的数据拼接为完整数据
- scatter：把一个GPU上的数据切分为多块，按照顺序向其他GPU发送数据块
- Reduce-Scatter：先reduce再做scatter
- AlltoAll：在所有rank之间进行数据的全量交换与重排。每个rank将其数据切分成若干块，分别发送给其他所有rank，同时从其他所有rank接收数据块。经过该操作后，每个rank的数据形状保持不变，但内容由来自所有rank的数据块拼接而成，实现了数据的全局重排

总述

- 张量并行 (TP)： 在单台服务器内部的GPU之间使用，以利用高速的NVLink互联。
- 流水线并行 (PP)： 在多台服务器之间使用，以切分模型的深度。
- 数据并行 (DP)： 在整个集群的所有设备上应用，以扩大总批次大小，加速收敛。
- 专家/序列/上下文并行 (EP/SP/CP)： 根据模型架构（是否为MoE）和应用场景（是否为长上下文）选择性地加入，作为第四个维度

数据并行 - DP

- 模型复制，数据分片：即每个设备（拥有完整的模型参数）独立处理一个 mini-batch 的数据
- 流程：
    1. 全局批次数据切分并分发给对应的GPUs
    2. 前向传播：每个GPU使用其模型副本独立计算其对应mini-batch数据的前向传播，计算loss
    3. 反向传播：每个GPU独立计算梯度
    4. **梯度同步**：通过all-reduce聚合梯度，确保每个GPU都获得全局梯度
    5. 权重更新：每个GPU使用同步后的全局梯度独立更新本地模型参数，从而保证所有本地模型一致
- 优点：实现简单，适用于大多是模型架构，扩展性好
- 缺点：
    - 无法解决模型太大，单卡放不下的问题
    - 随着GPU数量增加，all-reduce操作的通信开销成为主要瓶颈
    - 扩展效率受通信带宽和延迟影响
- ZeRO通过在不同设备间分割和管理模型状态来克服DP的内存冗余问题

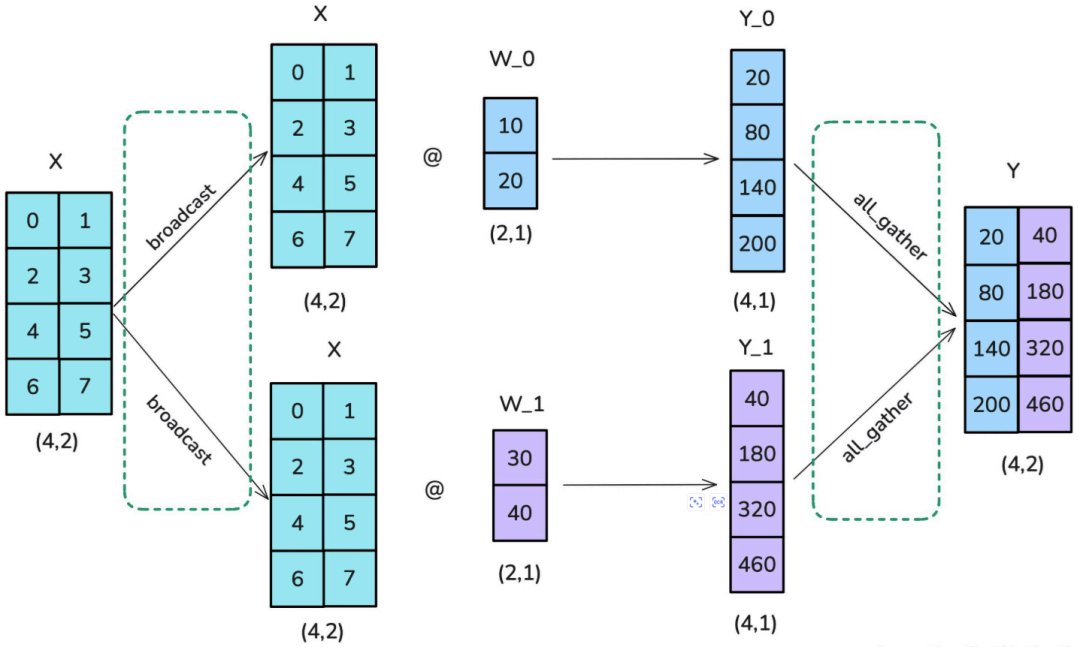
    - ZeRO-1:分割优化器状态，约节省30-40%的内存
    - ZeRO-2:分割优化器状态 + 梯度，约节省50-60%的内存
    - ZeRO-3:分割优化器状态 + 梯度 + 模型参数，约节省70-80%的内存
- ZeRO-DP使得数据并行也能用于训练单卡无法容纳的大模型

张量并行 - TP

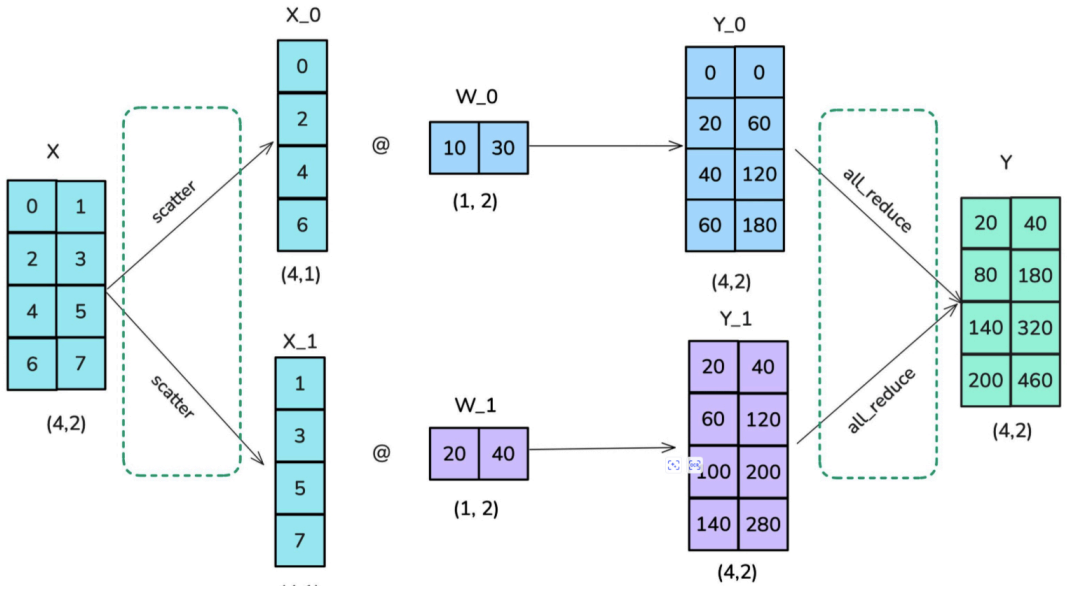
$$A \cdot B = A \cdot \begin{bmatrix} B_1 & B_2 & \cdots \end{bmatrix} = \begin{bmatrix} AB_1 & AB_2 & \cdots \end{bmatrix}$$

$$A \cdot B = \begin{bmatrix} A_1 & A_2 & \cdots \end{bmatrix} \begin{bmatrix} B_1 \\ B_2 \\ \vdots \end{bmatrix} = \sum_{i=1}^{n} A_i B_i$$

- 解决的问题（解决单层网络计算量过大或参数过多无法在单个设备上处理的问题）：ZeRO虽然极大节省了静态数据（模型状态：优化器状态、梯度、模型参数）的显存，但它无法压缩动态产生的中间计算结果（激活值），当激活值显存占用超过可用显存便会遇到新的问题
    - 激活值显存占用大小受：seq len和batch size影响
- TP是一种分片权重、梯度、优化器状态和激活值，而且无需在计算之前收集全部数据的方法
- TP：沿特定维度将张量分割为N个分片，并分布到N个GPU上
    - 按列分割：将完整输入通过广播的方式复制到每个GPU，然后将模型权重按列分割，将输入与GPU本地的部分权重计算，最后使用all-gather合并结果



    - 按行分割：将完整输入通过scatter的方式将输入的列分片按顺序发送给GPU，然后将模型权重按行分割，将输入与GPU本地的部分权重计算，最后使用all-reduce聚合结果



- 优点：
  - 减少单个GPU的显存占用（包括模型参数和激活值）
  - 计算粒度细，可以更好利用GPU并行能力
- 缺点：
  - 通信开销大切频繁，如果多节点会由于通信开销导致GPU实际每秒浮点运算大幅下降
  - 实现复杂度高
- 优化：
  - 使用Ring-AllReduce替换传统的All-Reduce，减少通信轮数和带宽需求
  - 将TP和PP结合，平衡计算和通信开销
  - 使用NVLink等高速互联技术
- 常用场景：
  - 与ZERO-R一起使用时，TP可以降低超大型模型的激活值内存占用
  - 对于小模型，当激活值内存占用不是问题时，TP可以避免DP由于batch size过大导致的收敛慢的问题

流水线并行 - PP

- 将模型的不同层/块分配到不同的GPU，在一个GPU上完成部分层计算后将输出传递到下一个GPU继续计算
- 流程：
  - 最朴素的就是一个批次的数据依次通过所有GPU，这会导致严重的bubble，即任何时刻只有一个GPU工作
  - Micro-batch Pipeline（解决气泡问题）：将一个大批次数据切割为多个小批次数据，将这些小批次数据依次送入pipeline，当第一个批次在GPU2执行时，第二个批次的数据就可以在GPU1执行
- 优点：
  - 降低单个GPU显存占用
  - 通信开销较小，只发生在pipeline中两个相邻阶段
  - 通信内容主要为激活值而为模型权重
  - 实现简单
- 缺点：

    - 存在bubble时间，尤其是启动和排空阶段
    - 要求对模型合理切分，以实现GPU负载均衡
    - 最慢的阶段会影响整体吞吐量
- 优化：
    - 增加pipeline深度，减少bubble比例
    - 重叠计算

长序列并行策略

解决由于序列过长导致的激活值显存爆炸问题

- 激活值显存占用约等于：层数 x 批次大小 x 序列长度的平方（NxN的注意力矩阵）

序列并行 - SP

- 在序列维度计算独立的模块（LayerNorm、Dropout、MLP中的逐点操作）的输入激活值切割
    - LayerNorm：对单个样本的最后一个维度（即特征维度）进行归一化
    - Dropout：为序列中的每一个token单独生成一个随机的“掩码”决定第 i 个token是否被“丢弃”
    - MLP中的逐点操作（非线性激活函数）：计算 output[i] = GELU(input[i]) 时，结果仅由 input[i] 的值决定，与 input 中的其他任何元素都无关
- 流程：
    1. 在进入非并行化模块前，被分割的激活值需要通过all-gether操作在序列维度拼接回完整的张量
    2. 在进入并行化模块前，输出的激活值被分割或者通过reduce-scatter操作将计算后的张量分散到各个设备

上下文并行 - CP（DeepSpeed的ZeRO-CP扩展支持）

- 是一种更激进的模式，将输入和所有的中间激活值都沿序列维度分割
- 流程：
    1. 输入分割：将长度为S的序列分割为N份，每个GPU处理长度为S/N的子序列
    2. 本地QKV计算：每个GPU根据自己的子序列局部QKV
    3. 全局KV同步：在注意力计算前，所有GPU需要执行依次all-gather操作，将本地的KV广播给其余GPU，这样每个GPU就拥有完整的KV和局部的Q
    4. 每个GPU并行根据局部Q和全局KV计算注意力输出
    5. 反向传播：类似模式，设计reduce-scatter操作

专家并行 - EP（稀疏化并行策略）

- MoE：用一组稀疏激活的“专家”网络（通常是FFN层）替换了传统Transformer中的密集FFN层。对于每个输入的token，一个可学习的“门控网络”（Gating Network）或“路由器”（Router）会动态地选择一小部分（如Top-2）专家来处理它 。

    - 模型的总参数量可以非常巨大（所有专家参数之和），但每个token的实际计算量（FLOPs）却保持不变或仅少量增加
- EP为将不同的专家分配到不同的GPU上
- 流程：
    1. 专家分发：将E个专家分配到N个GPU上，每个GPU有E/N个专家
    2. 路由计算：所有GPU上的token首先通过门控网络计算出该被发送到哪些专家
    3. All-to-All：最关键也是最贵的步骤，所有GPU参与依次All-to-All操作，将每个token发送到对应的目标专家所在的GPU
    4. 专家计算
    5. 第二次All-to-All：计算完成后，通过All-to-All将处理结果发送回token对应的原始GPU